# DBMON-99: Best Practice Summary

> **Series:** DBMON — Database Monitoring | **Notebook:** 7 of 7 | **Created:** March 2026 | **Last Updated:** 11/13/2026

## Overview

This notebook consolidates every actionable best practice for Dynatrace database monitoring extracted from the DBMON series (notebooks 01 through 06). It covers SQL databases, NoSQL databases, caches, message brokers, query analysis, dashboards, and alerting. Each best practice is definitive: it specifies exactly what to configure, what threshold to set, and what priority to assign.

---

## Table of Contents


1. OneAgent Instrumentation
2. ActiveGate Extensions
3. Span Attribute Coverage
4. SQL Database Monitoring
5. NoSQL Database Monitoring
6. Cache Monitoring (Redis / Memcached)
7. Message Broker Monitoring (Kafka / RabbitMQ)
8. Search Engine Monitoring (Elasticsearch)
9. Query Analysis and Optimization
10. Dashboards and KPIs
11. Alerting Thresholds
12. SLO Definitions
13. DQL Query Standards
14. Where to Go Deeper — Topic Series Map


---

## Prerequisites

| Requirement | Details |
|---|---|
| **Dynatrace Environment** | Dynatrace SaaS with Grail (Managed has no Grail, so these DQL cells do not run there) |
| **OneAgent** | Deployed on all application hosts making database, cache, and messaging calls |
| **ActiveGate** | Environment ActiveGate for Extensions 2.0 (remote DB metrics), or — SQL extensions only, Dynatrace 1.346+ (staged rollout; verify it has reached your tenant) — SQL Extension Executor on Kubernetes via Dynatrace Operator 1.8+ |
| **Permissions** | `storage:spans:read`, `storage:entities:read`, `storage:metrics:read` |
| **Prior Reading** | DBMON-01 through DBMON-06 for full context |

## 1. OneAgent Instrumentation

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 1 | Deploy OneAgent on every host that makes database calls | OneAgent installed on all application servers, not just database servers | **Critical** | Deployment |
| 2 | Enable deep code-level instrumentation | OneAgent auto-instruments JDBC, ADO.NET, and native database drivers by default; do not disable | **Critical** | Deployment |
| 3 | Verify `db.system` attribute is populated | Run `fetch spans, from:-1h \| filter isNotNull(db.system) \| summarize count(), by:{db.system}` — every expected technology must appear | **Critical** | Validation |
| 4 | Verify `db.query.text` capture | Confirm statements appear in `db.query.text` (parameterized SQL keeps its placeholders; `WHERE`-clause literals are masked). It is not on every DB span — 58% of SQL Server and 0% of Redis spans on the validation tenant — so check coverage before grouping on it | **Critical** | Validation |
| 5 | Confirm `db.namespace` is present | Run `filter isNotNull(db.namespace)` on spans; missing values indicate driver-level gaps | **Recommended** | Validation |

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 6 | Ensure `server.address` and `server.port` resolve | These fields must contain the actual database endpoint, not `localhost` or `127.0.0.1` when the DB is remote | **Recommended** | Validation |

## 2. ActiveGate Extensions

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 7 | Build new database integrations on Extensions 2.0 | Extensions 2.0 is the current extensions framework; Extensions Framework 1.0 reached end of support on 2025-03-31 (Python EF1.0: 2024-10-31). JMX and PMI EF1.0 extensions were the exception, supported past March 2025 as deprecated; they now have their own end-of-support date of **July 1, 2027** for SaaS environments. The SQL / Prometheus / SNMP data sources these extensions use run on an **Environment (host-based) ActiveGate**, the working path; SQL extensions (Oracle, SQL Server, PostgreSQL, MySQL/MariaDB, SAP HANA, Db2, Generic JDBC) can alternatively run on **SQL Extension Executor pods in Kubernetes** via Dynatrace Operator 1.8+, from Dynatrace 1.346 — a staged rollout, so verify it has reached your tenant; until then the Environment ActiveGate remains the working path | **Critical** | Deployment |

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 7b | Extensions are YAML-defined with optional Python data sources | YAML package executed by EEC; built-in SQL/Prometheus/SNMP data sources cover most cases; Python data source is for custom logic only — not required by default | **Recommended** | Architecture |
| 7c | Route extension logs to the `default_database_monitoring` bucket | Default destination for extension log output; reference this bucket name in IAM policies for DB-team access scoping (see ORGNZ-02 + IAM-04/05) | **Recommended** | IAM/Bucket |
| 8 | Install PostgreSQL extension | Captures: connections, transactions/sec, tuple operations, table/index sizes, lock waits, replication lag | **Recommended** | Extension |
| 9 | Install MySQL extension | Captures: connections, queries/sec, buffer pool hit ratio, InnoDB row ops, slow queries | **Recommended** | Extension |
| 10 | Install MS SQL Server extension | Captures: batch requests/sec, page life expectancy, buffer cache hit ratio, deadlocks, wait stats | **Recommended** | Extension |
| 11 | Install Oracle extension | Captures: sessions, tablespace usage, SGA/PGA memory, redo log waits, library cache hit ratio | **Recommended** | Extension |
| 12 | Install MongoDB extension | Captures: connections, operations/sec, document metrics, replica set health, storage engine stats | **Recommended** | Extension |
| 13 | Set extension polling interval to 60 seconds | Default polling; do not exceed 300s or real-time visibility degrades | **Recommended** | Configuration |

### Dynatrace Databases App

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 13b | Use the Databases app for first-pass triage | The docs list vendors *"such as PostgreSQL, MySQL, MSSQL, Oracle, MariaDB, and HanaDB"*, and *"Health highlights instances with critical, warning, or no active alerts"* — check it before running manual DQL analysis (per instance, the docs also define a weighted 0–100 health score) | **Recommended** | Triage |
| 13c | Use the app's execution-plan visualization as a first pass, not a replacement | Normalizes execution plans across PostgreSQL, MySQL, and SQL Server; fall back to the ActiveGate extension metrics above (rows 8-11) and the DQL patterns in DBMON-02/05 for deep-dive analysis the app's UI doesn't cover | **Recommended** | Diagnostics |

> **Sources:** Extensions (DT docs) — the Extensions 2.0 framework and the EEC execution model, Extension data sources (DT docs) — the built-in SQL / Prometheus / SNMP data sources and the optional Python data source, Run SQL extensions on Kubernetes (DT docs), Enable Dynatrace SQL database extensions (DT docs), What's new in Dynatrace SaaS 1.337 (DT docs) — the `default_database_monitoring` bucket, End-of-support announcements (DT docs) — lists *"Dynatrace Extension Framework 1.0"* under the 2025-03-31 end-of-support date and *"Dynatrace Extension Framework 1.0 (Python 3.8)"* under 2024-10-31, with the note *"Note that JMX and PMI Extensions Framework 1.0 are supported past March 2025 but are deprecated"*, re-read 09/28/2026, EF1 JMX and PMI extensions end of support (DT docs) — *"As of July 1, 2027, all Extension Framework 1.0 JMX and PMI extensions will be out of support for SaaS Environments"*, Database health score overview (DT docs) — *"The health score is the weighted average of all checks."*

## 3. Span Attribute Coverage

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 14 | Validate all six core DB span attributes | Every database span must carry: `db.system`, `db.query.text`, `db.operation.name`, `db.namespace`, `server.address`, `server.port` | **Critical** | Data Quality |

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 15 | Check for NULL fields periodically | Run `filter isNotNull(db.operation.name)` — NULL can mean the instrumentation carries the command in another field ( `code.function` for OneAgent MongoDB, `span.name` for OTel Redis) rather than a driver fault; check those before investigating the driver version | **Recommended** | Data Quality |
| 16 | Verify span.kind is client for outgoing DB calls | All database spans from the calling application must have `span.kind == "client"` (lowercase) | **Recommended** | Data Quality |
| 17 | For Kafka, verify `messaging.system` attribute | Kafka spans use `messaging.system` (not `db.system` ); confirm `messaging.destination.name` , `messaging.operation.type` , and `messaging.consumer.group.name` are populated | **Critical** | Data Quality |
| 18 | For RabbitMQ, verify `messaging.system` attribute | RabbitMQ spans use `messaging.system` (not `db.system` ); confirm `messaging.destination.name` is populated | **Critical** | Data Quality |

## 4. SQL Database Monitoring

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 19 | Set slow query threshold for SQL databases | **500ms** — any SQL query exceeding `duration > 500ms` is classified as slow | **Critical** | Threshold |
| 20 | Track read/write ratio | Classify SQL from the statement text, not `db.operation.name` : batched writes are named after their first keyword ( `SET` ), `PREPARE` / `RESULTSET` repeat a statement's text, and `CONNECT` / `COMMIT` are not statements | **Recommended** | Analysis |
| 21 | Rank queries by total execution time, not average | Sort by `total_time_ms = sum(duration)` to find highest-impact patterns regardless of individual speed | **Critical** | Optimization |

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 22 | Monitor response time distribution using latency tiers | Bucket into: `<1ms`, `1-10ms`, `10-100ms`, `100ms-1s`, `>1s` — track tier distribution over time | **Recommended** | Analysis |
| 23 | Baseline P50, P95, P99 over 24 hours | Run hourly percentile query on `duration` for each `db.system`; this is your performance reference point | **Critical** | Baseline |
| 24 | Monitor vendor-specific patterns | PostgreSQL: vacuum/lock contention; MySQL: buffer pool/replication lag; MSSQL: page life expectancy/deadlocks; Oracle: tablespace/SGA | **Recommended** | Vendor |
| 25 | Track errors by the recorded exception | Group `span.status_code == "error"` spans by the exception recorded in `span.events` (`exception.type`, `exception.message`) to distinguish connection timeouts, pool exhaustion, deadlocks, and constraint violations; `span.status_message` is experimental and often empty | **Critical** | Error Handling |

## Sprint-1.338 Update (May 2026)

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 25b | Use DQL `timeseries` for table/index metrics — classic ingest is blocked | Classic ingest is blocked for `postgres.tables.*`, `postgres.indexes.*`, `mysql.tables.*` and `mysql.indexes.*` (SaaS 1.338) — *"Ingested metrics with these keys are only available in Grail."* Query them with DQL `timeseries`. | **Critical** | API/Migration |

> **Sources:** What's new in Dynatrace SaaS 1.338 (DT docs) — *"To prevent dimension cardinality explosion for the latest versions of the database extensions"*, and *"Ingested metrics with these keys are only available in Grail."*

*In community practice these thresholds are the values teams converge on as starting points — Dynatrace publishes none of them. Treat each as a first guess to tune against your own baseline, not a recommended setting.*

## 5. NoSQL Database Monitoring

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 26 | Set slow query threshold for NoSQL databases | **100ms** for MongoDB; **300ms** critical threshold for all NoSQL | **Critical** | Threshold |
| 27 | Monitor MongoDB by collection | Group by `db.collection.name` in addition to `db.namespace` — collection-level granularity reveals hot spots | **Critical** | MongoDB |
| 28 | Track DynamoDB Scan-to-Query ratio | `Scan` operations read every item in the table; ratio must be Query-dominated; any high Scan count signals missing Global Secondary Indexes | **Critical** | DynamoDB |
| 29 | Set Cassandra slow threshold at 200ms | `filter duration > 200ms` — Cassandra operations exceeding 200ms indicate partition hotspots or cross-DC reads | **Recommended** | Cassandra |
| 30 | Monitor Cosmos DB by operation and error rate | Track `ReadItem`, `CreateItem`, `Query`, `ReplaceItem` separately; errors indicate RU throttling (HTTP 429) | **Recommended** | Cosmos DB |
| 31 | Classify NoSQL read/write operations correctly | Use `coalesce(db.operation.name, code.function)` (OneAgent MongoDB leaves `db.operation.name` empty); READ: `find`, `count`, `aggregate`, `getMore`, `Query`, `Scan`, `GetItem`, `SELECT`, `ReadItem`, `get`, `search`; WRITE: an explicit insert/update/delete/put list; anything else OTHER | **Recommended** | Analysis |
| 32 | Run cross-database comparison | Compare `total_calls`, `avg_ms`, `p95_ms`, `error_rate_pct` across all NoSQL systems in a single query for unified health view | **Recommended** | Analysis |

# 6. Cache Monitoring (Redis / Memcached)

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 33 | Set Redis slow command threshold at 5ms | `filter duration > 5ms` — Redis operations should complete in microseconds; anything over 5ms is abnormal | **Critical** | Threshold |
| 34 | Monitor Redis read/write balance | READ: `GET`, `MGET`, `HGET`, `HGETALL`, `LRANGE`, `SMEMBERS`, `ZRANGE`; WRITE: `SET`, `MSET`, `HSET`, `HMSET`, `LPUSH`, `RPUSH`, `SADD`, `ZADD`, `DEL`, `EXPIRE`; read the command from `coalesce(db.operation.name, span.name)` | **Recommended** | Analysis |
| 35 | Alert on Redis latency spikes | Track `p95_us` (microseconds) over time at 5m intervals; any sustained increase above 5000us (5ms) triggers investigation | **Critical** | Alerting |
| 36 | Watch for expensive Redis commands | `KEYS *`, `SMEMBERS` on large sets, `LRANGE` with large ranges cause latency spikes — flag any occurrence | **Critical** | Anti-Pattern |
| 37 | Report cache metrics in microseconds, not milliseconds | Redis and Memcached latency is sub-millisecond; use `avg(duration) / 1us` for microsecond precision (dividing a duration by a plain number leaves it a duration) | **Recommended** | Reporting |

## .NET Redis Client Coverage — OneAgent 1.337 and 1.343

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 37b | Confirm StackExchange.Redis instrumentation on .NET workloads | OneAgent 1.337 added StackExchange.Redis coverage for .NET applications. Verify your .NET services produce `db.system == "redis"` spans. | **Recommended** | Coverage |

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 37c | Confirm ServiceStack.Redis instrumentation on .NET workloads | Forthcoming / rolling out with **OneAgent 1.343** (released 07/28/2026): tracing support added for **ServiceStack.Redis** in .NET applications. Rollout is **per agent fleet, not per tenant** — **verify the OneAgent version installed on the hosts concerned** (the tenant version is not the agent version), then confirm those services produce `db.system == "redis"` spans. On **OneAgent 1.342 and earlier** only StackExchange.Redis is auto-instrumented, and ServiceStack.Redis calls produce no database spans | **Recommended** | Coverage |

*In community practice these thresholds are the values teams converge on as starting points — Dynatrace publishes none of them. Treat each as a first guess to tune against your own baseline, not a recommended setting.*

## 7. Message Broker Monitoring (Kafka / RabbitMQ)

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 38 | Track Kafka throughput by topic | Group by `messaging.destination.name` and `messaging.operation.type` (Dynatrace's dictionary: `publish`, `receive`, `process`, `peek`; newer OpenTelemetry producers send `send`) at 5m intervals | **Critical** | Kafka |
| 39 | Monitor Kafka consumer group processing latency | Group by `messaging.consumer.group.name` and `messaging.destination.name`; track `avg_ms` and `p95_ms` per group | **Critical** | Kafka |
| 40 | Detect Kafka consumer errors | Filter `span.status_code == "error"` on `messaging.operation.type == "process"` spans; any sustained errors indicate poisoned messages or deserialization failures | **Critical** | Kafka |
| 41 | Track RabbitMQ publish/consume rate per queue | Group by `messaging.destination.name` and `messaging.operation.type`; a publish rate exceeding consume rate signals backpressure | **Critical** | RabbitMQ |
| 42 | Monitor RabbitMQ message throughput trend | Use `makeTimeseries` at 5m intervals grouped by `messaging.operation.type` to detect throughput drops | **Recommended** | RabbitMQ |

## 8. Search Engine Monitoring (Elasticsearch)

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 43 | Set Elasticsearch slow query threshold at 200ms | `filter duration > 200ms` for search operations; indexing operations use a separate threshold | **Recommended** | Threshold |
| 44 | Break down Elasticsearch operations by type | Track `db.operation.name` (search, index, bulk, delete) separately; search latency and indexing throughput have different SLOs | **Recommended** | Analysis |
| 45 | Include OpenSearch in all Elasticsearch queries | Always filter `in(db.system, {"elasticsearch", "opensearch"})` — treat both identically | **Recommended** | Compatibility |

## 9. Query Analysis and Optimization

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 46 | Detect N+1 queries | Group spans by `trace.id` and `db.query.text`; any pattern with `calls_per_trace >= 10` is an N+1 candidate; >= 50 is confirmed N+1 | **Critical** | Anti-Pattern |
| 47 | Fix N+1 by batching | Replace N individual queries with a single batch query using `IN` clauses or JOINs | **Critical** | Remediation |
| 48 | Detect missing indexes heuristically | SELECT queries with `call_count >= 50` AND `avg_ms > 10` AND high `stddev_ms / avg_ms` ratio are index candidates | **Recommended** | Optimization |
| 49 | Detect queries getting slower over time | Run 24h `makeTimeseries` of `avg(duration)` by `db.system` at 1h intervals; rising trend signals index degradation or table growth | **Recommended** | Trend |
| 50 | Flag dynamic SQL / poor parameterization | Count one-off query patterns ( `call_count == 1` ); high count per `db.system` means queries are not parameterized, preventing plan caching | **Recommended** | Anti-Pattern |
| 51 | Prioritize optimization by total time impact | Sort query patterns by `total_time_ms = sum(duration)` , not by `avg_ms` — a fast query called 1M times has more impact than a slow query called once | **Critical** | Optimization |

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 52 | Use the tail ratio to detect outliers | Calculate `p99_ms / p50_ms`; a ratio above 10 indicates severe tail latency requiring investigation | **Recommended** | Analysis |
| 53 | Monitor connection pool pressure | Track `countIf(span.status_code == "error")` grouped by `dt.service.name` and `server.address`; connection refused or timeout errors indicate pool exhaustion | **Critical** | Connection Pool |
| 54 | Track error rate trend over 6h at 5m intervals | Use `makeTimeseries` with `total` and `errors` counts; a rising error trend within 30 minutes demands immediate action | **Critical** | Error Handling |

## 10. Dashboards and KPIs

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 55 | Build a unified health overview tile | Single query: group by `db.system`, show `total_calls`, `avg_ms`, `p95_ms`, `error_rate_pct`, `slow_rate_pct` | **Critical** | Dashboard |
| 56 | Include a "total database calls" single-value tile | `fetch spans, from:-1h \| filter isNotNull(db.system) \| summarize total_db_calls = count()` | **Critical** | Dashboard |
| 57 | Add "top 10 heaviest services" table tile | Group by `dt.service.name` with `call_count`, `avg_ms`, `error_count` (`dt.entity.service` is deprecated in the semantic dictionary) | **Critical** | Dashboard |
| 58 | Show P50 vs P95 vs P99 trend chart | 6h timeseries at 5m intervals; three lines on one chart reveals tail latency divergence | **Critical** | Dashboard |
| 59 | Add read vs write ratio trend tile | Classify operations into READ / WRITE / OTHER (rule 20); 6h timeseries at 5m intervals | **Recommended** | Dashboard |
| 60 | Add queries-per-minute trend by db.system | 6h timeseries at 1m intervals; used for throughput anomaly detection | **Recommended** | Dashboard |
| 61 | Add today-vs-yesterday volume comparison | Use `append` pattern: `from:-24h` for today, `from:-48h, to:-24h` for yesterday | **Recommended** | Dashboard |
| 62 | Include slow query detail table | Last 15m, `duration > 500ms`, fields: start_time, db.system, db.namespace, db.query.text, duration_ms, service_name | **Critical** | Dashboard |

# 11. Alerting Thresholds

## Error Rate Alerts

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 63 | Alert on error rate > 5% for 5 minutes | Severity: **Critical** — immediate page | **Critical** | Alert |
| 64 | Alert on error rate > 1% for 15 minutes | Severity: **Warning** — notification | **Critical** | Alert |
| 65 | Alert on any connection refused error | Severity: **Critical** — immediate page; zero tolerance for connection failures | **Critical** | Alert |
| 66 | Alert on deadlocks from the engine side | Severity: **Warning** — notification; use the Databases app's lock and wait metrics, since spans see a deadlock only as the victim session's error (DBMON-06) | **Recommended** | Alert |

## Slow Query Alerts (P95 Thresholds)

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 67 | SQL Warning threshold | P95 > **200ms** over 5-minute window | **Critical** | Alert |
| 68 | SQL Critical threshold | P95 > **500ms** over 5-minute window | **Critical** | Alert |
| 69 | NoSQL Warning threshold | P95 > **100ms** over 5-minute window | **Critical** | Alert |
| 70 | NoSQL Critical threshold | P95 > **300ms** over 5-minute window | **Critical** | Alert |
| 71 | Cache (Redis/Memcached) Warning threshold | P95 > **5ms** over 5-minute window | **Critical** | Alert |
| 72 | Cache (Redis/Memcached) Critical threshold | P95 > **20ms** over 5-minute window | **Critical** | Alert |
| 73 | Elasticsearch Warning threshold | P95 > **500ms** over 5-minute window | **Recommended** | Alert |
| 74 | Elasticsearch Critical threshold | P95 > **2000ms** over 5-minute window | **Recommended** | Alert |

### Throughput Alerts

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 75 | Alert on throughput drop > 50% vs 7-day average | Compare current queries/min to 7-day baseline; sudden drop indicates upstream failure or connectivity loss | **Critical** | Alert |
| 76 | Alert on slow query count > 10 per 5-minute window | Sustained slow query volume indicates systemic degradation, not a one-off outlier | **Recommended** | Alert |

## 12. SLO Definitions

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 77 | Define availability SLO | **99.9%** of database calls succeed, measured over a 24h rolling window as `count() - countIf(span.status_code == "error")` — `span.status_code` is null on successful spans, so a `!= "error"` comparison counts nothing | **Critical** | SLO |
| 78 | Define latency SLO | **95%** of calls complete under the vendor-specific P95 threshold (SQL: 500ms, NoSQL: 300ms, Cache: 20ms, Search: 2000ms) | **Critical** | SLO |
| 79 | Define throughput SLO | No more than **50% drop** from 7-day baseline queries/min | **Recommended** | SLO |
| 80 | Measure SLO compliance hourly | Run `makeTimeseries` at 1h intervals with `total` and `errors` / `under_threshold` counts per `db.system` | **Critical** | SLO |
| 81 | Track SLO budget burn rate | If 24h availability drops below 99.9%, calculate remaining error budget: `(1 - 0.999) * total_calls - error_count` | **Recommended** | SLO |

## 13. DQL Query Standards

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 82 | Always specify a time range on fetch | `fetch spans, from:-1h` — never omit `from:` on span queries; default 2h scan is excessive | **Critical** | DQL |

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 83 | Filter on `isNotNull(db.system)` first | This is the primary filter to isolate database spans; apply immediately after `fetch` | **Critical** | DQL |
| 84 | Use named parameters in functions | `round(value, decimals: 2)`, `if(cond, then: "x", else: "y")` — positional parameters cause errors | **Critical** | DQL |
| 85 | Alias all aggregations | `summarize c = count()`, not `summarize count()` — unaliased aggregations get a generated name ( `count()` ) that must be backtick-quoted to reference; `sort count() desc` without backticks fails | **Critical** | DQL |
| 86 | Use duration arithmetic (NOT nanosecond constants) | Write `duration / 1ms`, `duration / 1s`, `(t2-t1) / 1m` — divide a duration by another duration to get a unitless number. Never `duration / 1000000.0`; never `toLong(duration) / 1ms` (long/duration fails type-check). Per `dt-dql-essentials`. | **Critical** | DQL |
| 87 | Show service names, not IDs | On spans, group by `dt.service.name` (stable). Where you hold a Smartscape ID, `getNodeName(dt.smartscape.service)` resolves it; `entityName(dt.entity.service, …)` is the classic form and `dt.entity.service` is deprecated. Never show raw entity IDs in dashboards. | **Recommended** | DQL |
| 88 | Filter early, sort last | Apply `filter` immediately after `fetch`; apply `sort` only after `summarize`; never `sort` right after `fetch` | **Critical** | DQL |
| 89 | Use `countIf()` for conditional aggregation | `errors = countIf(span.status_code == "error")` inside `summarize` — do not use separate filter + count | **Recommended** | DQL |
| 90 | Calculate error rate with safe division | `error_rate_pct = round((toDouble(error_count) / toDouble(total_calls)) * 100, decimals: 2)` — use `toDouble()` to avoid integer truncation | **Recommended** | DQL |

### Modern DQL standards (added 2026-05-07)

| # | Best Practice | Recommended Setting/Value | Priority | Category |
|---|---|---|---|---|
| 91 | Use `event.status` / `event.start` / `event.end` on `dt.davis.problems` | NOT `status` / `end_time` (recurring authoring bug). Status values are `"ACTIVE"` and `"CLOSED"` (not `"OPEN"`). For duration: `(event.end - event.start) / 1m` (NOT `event.end - timestamp` — `timestamp` is record-write time, not problem-open time). | **Critical** | DQL |
| 92 | Prefer `smartscapeNodes "<TYPE>"` for new entity queries | Modern Smartscape topology query: `smartscapeNodes "SERVICE"`, `smartscapeNodes "HOST"`, etc. Legacy `fetch dt.entity.<type>` still works on hybrid tenants but is deprecated. | **Recommended** | DQL |
| 93 | Wrap multiple `summarize` aggregations in `{}` | When summarizing 2+ aggregations, wrap them: `summarize { a = count(), b = avg(x) }, by: {field}`. Unwrapped aggregations combined with `by:` raise a `PARAMETERS_SHOULD_BE_GROUPED` INFO notification; without `by:` they raise none (tenant-verified 10/14/2026). The query runs either way — wrap them for consistency. | **Recommended** | DQL |

## 14. Where to Go Deeper — Topic Series Map

DBMON covers database monitoring foundations. These topic series cover adjacent domains in depth:

| Domain | Topic Series | Notebooks |
|---|---|---|
| **Distributed tracing & spans** (DB spans live here) | SPANS | 9 |
| **OpenPipeline log processing** | OPLOGS | 9 |
| **OpenPipeline beyond logs (metrics from spans)** | OPIPE | 8 |
| **Dynatrace Intelligence (Davis anomaly detection on DB metrics)** | AIOPS | 8 |
| **Workflows & alert notifications** (DB alert routing) | WFLOW | 12 |
| **Dashboard strategy & executive reporting** | DASH | 8 |
| **Bucket strategy** ( `default_database_monitoring` and custom DB buckets) | ORGNZ | 11 |
| **IAM administration** (bucket-scoped policies for DB-team access) | IAM | 16 |

| Domain | Topic Series | Notebooks |
|---|---|---|
| **Tagging strategy** (host-group + ownership tagging for DBs) | FAQ-01, FAQ-02 | 2+ |
| **Configuration automation** (Monaco/Terraform for extension deployment) | AUTOM | 14 |
| **OpenTelemetry integration** (OTel-instrumented DBs) | OTEL | 9 |
| **Cloud DB integrations** (RDS, Aurora, Cosmos DB, Cloud SQL) | CLOUD | 9 |

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*